

Методичка 8. Срезы строк и списков

1. Зачем нужна эта тема

Срезы нужны, чтобы быстро получать часть строки или списка.

Например, с помощью срезов можно:

- взять первые три буквы слова;
- взять последние четыре символа строки;
- убрать первый символ;
- убрать последний символ;
- развернуть строку наоборот;
- получить часть списка;
- проверить палиндром;
- заменить часть строки через создание новой строки.

Срезы часто используются в задачах на строки, списки, шифры, проверку слов и обработку данных.

Эта тема связана со строками и списками. Чтобы понимать срезы, нужно помнить, что у каждого символа строки и каждого элемента списка есть индекс.

2. Главная идея темы

Срез позволяет взять не один элемент, а сразу часть последовательности.

Общий вид:

```
sequence[start:stop:step]
```

Где:

- `start` — с какого индекса начать;
- `stop` — перед каким индексом остановиться;
- `step` — с каким шагом идти.

Важно: правая граница `stop` не включается.

Срез — получение части строки или списка.

Левая граница — индекс, с которого начинается срез.

Правая граница — индекс, перед которым срез останавливается.

Шаг — через сколько элементов двигаться.

3. Базовый синтаксис

Срез строки

```
word = "python"

print(word[0:2])
```

Результат:

```
py
```

Срез `word[0:2]` берёт символы с индексами 0 и 1. Индекс 2 не включается.

Срез от начала

```
word = "python"

print(word[:3])
```

Результат:

```
pyt
```

Если не указать начало, Python берёт с начала строки.

Срез до конца

```
word = "python"

print(word[3:])
```

Результат:

```
hon
```

Если не указать конец, Python берёт до конца строки.

Срез с шагом

```
word = "python"  
  
print(word[::2])
```

Результат:

```
pto
```

Берётся каждый второй символ.

Разворот строки

```
word = "python"  
  
print(word[::-1])
```

Результат:

```
nohtyp
```

Шаг `-1` означает движение справа налево.

Срез списка

```
numbers = [10, 20, 30, 40, 50]  
  
print(numbers[1:4])
```

Результат:

```
[20, 30, 40]
```

Срезы работают не только со строками, но и со списками.

4. Разбор темы от простого к сложному

Шаг 1. Взять первые символы

```
word = "program"

print(word[:3])
```

Результат:

```
pro
```

Так можно взять начало строки.

Шаг 2. Взять последние символы

```
word = "program"

print(word[-3:])
```

Результат:

```
ram
```

Отрицательные индексы считаются с конца строки.

Шаг 3. Убрать первый символ

```
word = "program"

print(word[1:])
```

Результат:

```
rogram
```

Срез начинается с индекса 1, поэтому первый символ пропускается.

Шаг 4. Убрать последний символ

```
word = "program"

print(word[:-1])
```

Результат:

```
progra
```

`-1` — это индекс последнего символа. Правая граница не включается, поэтому последний символ не попадает в результат.

Шаг 5. Взять середину строки

```
word = "program"

print(word[2:5])
```

Результат:

```
ogr
```

Берутся символы с индексами 2, 3, 4.

Шаг 6. Развернуть строку

```
word = "level"

reversed_word = word[::-1]

print(reversed_word)
```

Результат:

```
level
```

Срез `[::-1]` создаёт строку в обратном порядке.

Шаг 7. Проверить палиндром

```
word = input("Введите слово: ")

if word == word[::-1]:
    print("Палиндром")
else:
    print("Не палиндром")
```

Палиндром — это слово, которое читается одинаково слева направо и справа налево.

Например:

- `ТОПОТ` ;

- `казак` ;
- `level` .

Шаг 8. Получить часть списка

```
numbers = [1, 2, 3, 4, 5, 6]

first_part = numbers[:3]
second_part = numbers[3:]

print(first_part)
print(second_part)
```

Результат:

```
[1, 2, 3]
[4, 5, 6]
```

Срезы помогают делить список на части.

Шаг 9. Создать новую строку вместо изменения старой

```
word = "cat"

new_word = "b" + word[1:]

print(new_word)
```

Результат:

```
bat
```

Строки нельзя изменять по индексу, но можно собрать новую строку из частей.

5. Частые ошибки учеников

Ошибка 1. Думают, что правая граница включается

Неправильно:

```
word = "python"

print(word[0:3])
```

Почему это может быть ошибкой понимания:

Этот код выведет `pyt`, а не символы с индексами 0, 1, 2, 3. Правая граница не включается.

Правильно понимать так:

```
word = "python"

print(word[0:3]) # индексы 0, 1, 2
```

Ошибка 2. Путаются в отрицательных индексах

Неправильно:

```
word = "python"

print(word[-1])
```

Почему это может быть ошибкой понимания:

`word[-1]` — это не первый символ, а последний.

Правильно:

```
word = "python"

print(word[0])    # первый символ
print(word[-1])   # последний символ
```

Ошибка 3. Неправильно берут последние символы

Неправильно:

```
word = "python"

print(word[-3:-1])
```

Почему это ошибка:

Такой срез не включает последний символ. Он даст `ho`, а не `hon`.

Правильно:

```
word = "python"

print(word[-3:])
```

Ошибка 4. Забывают шаг при развороте

Неправильно:

```
word = "python"

print(word[-1:0])
```

Почему это ошибка:

Такой срез не развернёт строку. Для движения справа налево нужен отрицательный шаг.

Правильно:

```
word = "python"

print(word[::-1])
```

Ошибка 5. Путают срез и один элемент

Неправильно:

```
word = "python"

part = word[0]

print(part)
```

Почему это может быть не тем, что нужно:

`word[0]` возвращает один символ. Если нужна часть строки, нужен срез.

Правильно:

```
word = "python"

part = word[0:3]

print(part)
```


Ошибка 6. Пытаются изменить строку напрямую

Неправильно:

```
word = "cat"

word[0] = "b"
```

Почему это ошибка:

Строки нельзя изменять по индексу.

Правильно:

```
word = "cat"

word = "b" + word[1:]

print(word)
```

Ошибка 7. Неправильно проверяют палиндром

Неправильно:

```
word = input("Введите слово: ")

if word[::-1]:
    print("Палиндром")
```

Почему это ошибка:

`word[::-1]` просто создаёт перевёрнутую строку. Нужно сравнить её с исходной.

Правильно:

```
word = input("Введите слово: ")

if word == word[::-1]:
    print("Палиндром")
else:
    print("Не палиндром")
```

6. Мини-примеры для закрепления

Пример 1. Первые три символа

```
word = "python"

print(word[:3])
```

Программа выводит начало строки. Закрепляется срез от начала.

Пример 2. Последние три символа

```
word = "python"

print(word[-3:])
```

Программа выводит конец строки. Закрепляются отрицательные индексы.

Пример 3. Строка без первого символа

```
word = "python"

print(word[1:])
```

Закрепляется срез до конца.

Пример 4. Строка без последнего символа

```
word = "python"

print(word[:-1])
```

Закрепляется правая граница и отрицательный индекс.

Пример 5. Каждый второй символ

```
word = "python"

print(word[::2])
```

Закрепляется шаг среза.

Пример 6. Разворот строки

```
word = "python"

print(word[::-1])
```

Закрепляется отрицательный шаг.

Пример 7. Палиндром

```
word = input("Введите слово: ")

if word == word[::-1]:
    print("Да")
else:
    print("Нет")
```

Закрепляется сравнение строки с её разворотом.

Пример 8. Срез списка

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])
```

Закрепляется применение срезов к спискам.

7. Практические задания

Лёгкий уровень

1. Дана строка. Вывести первые 2 символа.
2. Дана строка. Вывести последние 2 символа.
3. Дана строка. Вывести строку без первого символа.
4. Дана строка. Вывести строку без последнего символа.
5. Дан список из 5 чисел. Вывести первые 3 элемента.

Средний уровень

1. Пользователь вводит слово. Вывести его наоборот.
2. Пользователь вводит слово. Проверить, является ли оно палиндромом.
3. Дана строка. Вывести каждый второй символ.
4. Дана строка. Поменять первую букву на `"*"`, создав новую строку.
5. Дан список чисел. Разделить его на две части: первые 3 элемента и остальные.

Повышенный уровень

1. Пользователь вводит строку. Проверить, одинаковые ли первые 3 и последние 3 символа.
2. Пользователь вводит слово. Создать новое слово без первой и последней буквы.
3. Пользователь вводит строку. Вывести её наоборот, но без последнего символа исходной строки.

8. Контрольные вопросы

1. Что такое срез?
2. Как записывается срез?
3. Что означает `start` ?
4. Что означает `stop` ?
5. Включается ли правая граница?
6. Что означает `step` ?
7. Что делает срез `[::-1]` ?
8. Как взять первые 3 символа строки?
9. Как взять последние 3 символа строки?
10. Можно ли использовать срезы со списками?
11. Почему строку нельзя изменить по индексу?
12. Как создать новую строку на основе старой?

9. Краткий конспект в конце

Главная идея темы: срезы позволяют получать часть строки или списка.

Основная форма:

```
sequence[start:stop:step]
```

Примеры:

```
word[:3]
```

```
word[3:]
```

```
word[-3:]
```

```
word[::-1]
```

```
numbers[1:4]
```

Важно запомнить:

- правая граница не включается;
- пустое начало означает “с начала”;
- пустой конец означает “до конца”;
- отрицательные индексы идут с конца;
- шаг `-1` разворачивает строку или список;
- строки нельзя изменять, но можно создавать новые строки из частей.

Частые ошибки:

- считать, что правая граница включается;
- путать отрицательные индексы;
- забывать шаг `-1` при развороте;
- пытаться изменить строку по индексу;
- не сравнивать строку с разворотом при проверке палиндрома.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- делать срезы строк;
- делать срезы списков;
- брать начало строки;
- брать конец строки;
- использовать отрицательные индексы;
- использовать шаг среза;
- разворачивать строку;
- проверять палиндром;
- создавать новую строку из частей старой.